

HTML, made simple

A visual, step-by-step revision guide for your first webpages.

Imagine a webpage as a school notebook page: a title, sections, pictures, links, and a form. HTML tells the browser what each part is. CSS decides how it looks. JavaScript decides what happens when someone interacts.

The whole webpage at a glance

Browser tab	<code><head></code> holds the title and page information.
Top area	<code><header></code> often holds the site name and <code><nav></code> links.
Main area	<code><main></code> holds headings, text, images, sections, and forms.
Bottom area	<code><footer></code> often holds contact or copyright details.

Follow this path

1. Skeleton	Create the basic HTML document.
2. Content	Add headings, paragraphs, images, links, and lists.
3. Structure	Group content and choose meaningful page areas.
4. Interaction	Build a form with labels, inputs, and buttons.
5. Practice	Build a whole page from the pieces.

01 Your first HTML document

First learn the container that holds everything else.

A document is a set of nested boxes. The outer box is html. Inside it are head and body. You normally see the body on the page.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
  </body>
</html>
```

DOCTYPE — Tells the browser to use modern HTML rules.

html — Wraps the entire page; lang says which language it uses.

head — Stores page information, usually not displayed inside the page.

title — Names the browser tab.

body — Holds the content you see on the page.

charset — Helps characters and symbols display properly.

viewport — Helps the page fit phone screens.

Picture it

html is a folder. head is its information card. body is the page a visitor reads.

02 Learn the tiny building block

Once tags make sense, the rest of HTML becomes much easier.

```
<p class="intro">Hello, Saby!</p>
```

`<p>` opens the paragraph. Hello, Saby! is its content. `</p>` closes it. Together they form one element. `class="intro"` is an attribute: extra information in the opening tag.

Picture it

An element is like a labeled box: the tag is the label, the content is inside, and attributes are extra instructions written on the label.

Nesting = a box inside another box

HTML elements often live inside other elements. Close the inner box before closing the outer box.

```
<div>
  <p>Welcome to <strong>HTML</strong>.</p>
</div>
```

div contains p; p contains strong. The parent contains its child. p and another p inside the same div would be siblings.

Try it

Point to the parent, child, and text in this example.

Some tags have no closing tag

An image or input is a single, empty element. These are called void elements.

```

<input type="text">
<br>
<hr>
```

Do not write `</img>` or `</input>`. `br` is a line break; `hr` separates topics. Do not use repeated `br` tags to create layout space.

03 Put words on the page

Use headings for the outline and paragraphs for sentences.

Headings and paragraphs

h1 is a top-level heading; h2 names a section beneath it; h3 can name a smaller subsection. p is regular paragraph text.

```
<h1>My Recipe Book</h1>
<h2>Breakfast</h2>
<h3>Oatmeal</h3>
<p>Cook oats with milk for five minutes.</p>
```

Think of h1 as the book title, h2 as a chapter, and h3 as a topic within that chapter. Choose heading levels by meaning, then use CSS for size.

Try it

Change “Breakfast” to “Dinner” and add another h3 recipe.

Give words meaning

strong means important text; em marks emphasis. They are often shown as bold and italic by browsers.

```
<p><strong>Warning:</strong> The pan is hot.</p>
<p>Read the instructions <em>carefully</em>.</p>
```

The choice expresses meaning, not just appearance.

04 Connect pages and show images

A link takes the reader somewhere. An image displays visual content.

Links

`a` is an anchor; `href` is its destination. The text between the tags is what the visitor clicks.

```
<a href="about.html">About our class</a>
<a href="https://example.com">Visit example.com</a>
<a href="#contact">Jump to contact</a>
<h2 id="contact">Contact</h2>
```

The last link jumps to the element whose `id` is `contact`. Use a link to navigate; use a button to perform an action.

Try it

Create a link to a second page named `contact.html`.

Images and alt text

`src` tells the browser where to find the picture. `alt` describes its meaning when someone cannot see it or it fails to load.

```

```

For purely decorative images, `alt=""` can be appropriate. “Image” is usually a poor description.

Where is the file?

A relative path starts from the current HTML file. `./` means this folder; `../` means one folder above.

```
project/
  index.html
  images/
    shoe.jpg
```

```

```

A complete URL such as `https://example.com/shoe.jpg` is an absolute web address.

05 Make groups and lists

Choose a list when you have a collection of related items.

Bullets and numbered steps

ul is an unordered list for items; ol is an ordered list when the order matters. Each item uses li.

```
<h2>Ingredients</h2>
<ul><li>Oats</li><li>Milk</li></ul>
<h2>Steps</h2>
<ol><li>Heat milk</li><li>Add oats</li></ol>
```

A shopping list fits ul. A recipe procedure fits ol.

Try it

Write your learning plan as three numbered steps.

Generic containers

div groups larger chunks without giving them a particular meaning. span groups a small piece of text. CSS can change how either looks.

```
<div class="product-card">
  <h2>Bag</h2>
  <p>Price: <span class="price">₹999</span></p>
</div>
```

class is a reusable label, such as product-card. An id is a unique label on the page, such as id="contact".

06 Give the page a clear structure

Semantic HTML tells people and browsers what a region is for.

Imagine a newspaper: it has a masthead, navigation, main stories, related stories, and a bottom area. HTML has names for these jobs.

```
<header>
  <h1>News Today</h1>
  <nav><a href="index.html">Home</a></nav>
</header>
<main>
  <section>
    <h2>Technology</h2>
    <article>
      <h3>Students build websites</h3>
      <p>Today our class built a page.</p>
    </article>
  </section>
  <aside><h2>Related stories</h2></aside>
</main>
<footer><p>© 2026 News Today</p></footer>
```

header — Introduction to a page or a part of it.

nav — Major navigation links.

main — The page's primary content.

section — A related group, generally with a heading.

article — A self-contained story, post, or review.

aside — Related but secondary information.

footer — Closing information for a page or section.

Remember

Use a `div` if you only need a styling wrapper. Use a semantic tag when its name accurately describes the content.

07 Collect information with forms

A form lets a visitor give the page information.

Think of a paper admission form: each question has a label, a place to write an answer, and a submit button. An HTML form has the same parts.

```
<form action="/register" method="post">
  <label for="student-name">Your name</label>
  <input id="student-name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <button type="submit">Register</button>
</form>
```

form — Contains the controls and specifies where/how to send them.

label — Tells the visitor what to enter; for connects to the input id.

input — The place where they enter or choose a value.

name — The key sent with the value, such as email=student@example.com.

required — Asks the browser to block an empty submission.

button — Starts an action; type="submit" submits the form.

Try it

Click the text “Email.” With the matching for and id, the browser focuses its input. Remove name="email" and notice that ordinary form submission will not include that field.

08 Pick the right form control

Different questions need different kinds of answers.

Common input types

Choose a type that matches the answer. Browsers may show a suitable keyboard or simple built-in checks.

```
<input type="text" name="fullName">
<input type="email" name="email">
<input type="password" name="password">
<input type="number" name="age" min="1">
<input type="date" name="birthday">
<input type="file" name="photo">
```

Always pair the real controls with labels. Placeholder text is only a hint, not a label.

Choose one or choose many

Radio buttons sharing the same name are one group. Checkboxes can be selected independently.

```
<fieldset>
  <legend>Delivery</legend>
  <label><input type="radio" name="delivery"
    value="standard"> Standard</label>
  <label><input type="radio" name="delivery"
    value="express"> Express</label>
</fieldset>
<label><input type="checkbox" name="updates"> Send updates</label>
```

fieldset and legend name the group of related questions.

Dropdown and long answer

select gives a set of choices; textarea allows several lines of text.

```
<label for="city">City</label>
<select id="city" name="city">
  <option value="indore">Indore</option>
  <option value="patna">Patna</option>
</select>
<label for="message">Message</label>
<textarea id="message" name="message"></textarea>
```

Try a registration form with name, email, city, and message.

09 Understand submission and helpful rules

These few details prevent common beginner mistakes.

GET usually adds form data to a URL, making it a natural choice for a search. POST sends form data in the request body, commonly used to submit data for processing. Use HTTPS and validate on the server in either case.

```
<form action="/search" method="get">
  <label for="q">Search</label>
  <input id="q" name="q" type="search">
  <button type="submit">Search</button>
</form>
```

If a student searches for shoes, the URL may look like `/search?q=shoes`. The action names the destination; the method controls how it is sent.

Three button behaviors

```
<button type="submit">Send</button>
<button type="reset">Clear fields</button>
<button type="button">Open menu</button>
```

Inside a form, a button without type normally acts as submit. State its type explicitly.

Common constraints

```
<input name="username" minlength="3" required>
<input name="quantity" type="number" min="1" max="10">
<input name="code" value="A123" readonly>
<input name="expired" value="OLD" disabled>
```

readonly means the visitor cannot edit the value but it may still submit; disabled generally cannot be used and is not submitted. Browser checks are helpful but do not replace server checks.

10 Useful extras you will meet

Recognize these when you see them; learn them deeply when a project needs them.

Tables = data with rows and columns

Use a table for marks, timetables, or similar data. th is a heading cell; td is a data cell.

```
<table>
  <caption>Marks</caption>
  <thead><tr><th scope="col">Name</th>
    <th scope="col">Score</th></tr></thead>
  <tbody><tr><td>Asha</td><td>92</td></tr></tbody>
</table>
```

tr creates a row. colspan and rowspan can merge cells. Do not use tables to lay out a whole webpage.

Audio, video, and embedded content

controls supplies playback controls. iframe places an external page inside yours when embedding is allowed.

```
<audio src="lesson.mp3" controls></audio>
<video src="demo.mp4" controls width="320"></video>
<iframe src="https://example.com/embed"
  title="Course video"></iframe>
```

Some websites block embedding. A useful iframe title explains what is inside.

Special characters and comments

Use character references for special symbols. Comments explain source code and are visible to someone viewing it.

```
<p>5 &lt; 10 &amp; 10 &gt; 5</p>
<p>&copy; 2026 My Class</p>
<!-- Main product list begins here -->
```

Never put passwords or secret keys in HTML or comments.

11 Make the page work for everyone

Good HTML helps people using a keyboard or screen reader.

Images: Describe the useful information with alt. A decorative image can use alt="".

Forms: Give every control a real label; do not depend on placeholder text.

Actions: Use a real button for an action and an anchor for navigation.

Headings: Keep the outline logical. Do not pick h1 only because it looks bigger.

Document: A specific title and understandable links make the page easier to find and use.

```
<title>Student projects | My Class</title>
<meta name="description"
      content="Browse projects built by our students.">

<a href="projects.html">See student projects</a>
```

Quick check

Could somebody understand this page with images turned off? Can you tab to every link, input, and button?

A tiny DOM preview

The browser reads HTML and creates a tree called the Document Object Model (DOM). JavaScript can later select and change parts of that tree.

```
body
├── main
│   ├── h1
│   └── form
│       ├── label
│       ├── input
│       └── button
```

12 Build one complete page

Put the pieces together before you begin styling.

Your challenge: build a student profile page with a header, navigation, one image, an about section, a skills list, a contact form, and a footer. Start with this example, then replace the content with your own.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Profile</title>
</head>
<body>
  <header>
    <h1>My Profile</h1>
    <nav><a href="#about">About</a>
      <a href="#contact">Contact</a></nav>
  </header>
  <main>
    <section id="about">
      <h2>About Me</h2>
      
      <p>I am learning to build websites.</p>
      <h3>Skills</h3>
      <ul><li>HTML</li><li>CSS</li></ul>
    </section>
    <section id="contact">
      <h2>Contact</h2>
      <form action="/contact" method="post">
        <label for="email">Email</label>
        <input id="email" name="email" type="email" required>
        <button type="submit">Send</button>
      </form>
    </section>
  </main>
  <footer><p>My first webpage</p></footer>
</body>
</html>
```

Check your understanding

Explain why each tag is there. Click both navigation links. Replace the image filename with your own.

Add a third skill. Then write the page once without looking.

13 One-page memory sheet

Use this page for quick revision before class or an interview.

A page — html contains head and body.

Content — h1-h6 = headings; p = paragraph; strong/em = meaning.

Go somewhere — a + href = link; img + src + alt = image.

Group things — ul/ol + li = lists; div/span = generic containers.

Page regions — header, nav, main, section, article, aside, footer.

Collect answers — form, label, input, select, textarea, button.

Send answers — action = destination; method = GET/POST; name = field key.

Accessible habits — alt, labels, meaningful links, native controls.

Extra tools — table for data; audio/video for media; iframe for embeds.

Can you answer these without looking?

- What is the difference between head and body?
- What do opening and closing tags do?
- When would you choose a link instead of a button?
- Why does an image need alt text?
- How does a label connect to an input?
- What is the difference between a class and an id?
- Why choose section instead of div?
- What does an input's name do when a form submits?
- Why are radio buttons with the same name a group?
- What is the DOM?

You are ready for CSS when

You can build the profile page's HTML on your own and explain what each part means. CSS will make it look polished; HTML gives it a strong foundation.